Bumper Technical Assignment

GuestBook Backend API Development Report

—

11.05.2026

Müberra BÜLBÜL ÖDÜN

INTRODUCTION

This document explains the technical details, architectural decisions, and requirement coverage of the GuestBook Backend API assignment.

The project was developed using Django and Django REST Framework with a focus on:

- clean code structure

- optimized ORM queries

- scalability considerations

- API validation

- pagination support

- test coverage

- maintainability

During development, special attention was given not only to fulfilling the assignment requirements, but also to implementing additional improvements such as query optimization, validation enhancements, automated tests, admin panel support, and code formatting standards.

This document explains step-by-step how each requirement was implemented and which technical decisions were made throughout the project.

Sincerely,

Müberra ÖDÜN

Technologies Used

The following technologies and tools were used during development:

Python	Main programming language used for backend development.
Django	Used as the main backend framework to provide project structure, ORM support, admin panel, and scalable architecture.
Django REST Framework (DRF)	Used to build RESTful APIs, serializer validation, pagination support, response handling, and API testing.
SQLite	Used as the database solution for storing users and guestbook entries.
Django ORM	Used for database operations and advanced query optimization techniques such as ***select_related***, ***annotate,*** and ***Subquery***.
Coverage	Used to measure automated test coverage and verify tested code quality.
Black Formatter	Used to standardize code formatting and improve code readability and consistency.

Project Structure

🔗 **Project Structure Screenshot: https://prnt.sc/jp4NgCD2LvSf**

Demo Video

The following demo video includes API requests, pagination behavior, unique user creation logic, admin panel overview, and response examples for all implemented endpoints.

🔗 **Demo Link: https://jam.dev/c/17fa0cc6-ad60-46d4-bbfa-469471bc2926**

Unit & E2E Test

🔗 **Test Screenshot: https://prnt.sc/LakkMH26qjBg**

Coverage

🔗 **Coverage Screenshot: https://prnt.sc/5rAbD1uVlzpS**

Project Structure

The project structure was organized by separating database operations, validations, business logic, pagination, and tests into different files.

Main components:

models.py

- Created for database table definitions.
- User and Entry models were added in this file.
- Foreign key relationships and database indexes were configured here.

serializers.py

- Added for request and response validation.
- Input validations such as blank field control and whitespace trimming were implemented here.
- Response structures for API outputs were also defined in this file.

views.py

- Used to handle API operations and business logic.
- Entry creation, entry listing, and user-based data operations were implemented here.
- Query optimizations such as `select_related`, `annotate`, and `Subquery` were added in this file.

pagination.py

- Created for custom pagination support.
- Pagination response structure and page size configuration were implemented here.

urls.py

- Added for API endpoint definitions.
- API routes were configured in this file.

tests.py

- Added for automated API testing.
- Endpoint tests and response validations were implemented here.

admin.py

- Configured for Django admin panel management.
- User and Entry models were registered here.
- Search, ordering, and filtering configurations were added for easier management.

settings.py

- Updated for project configuration.
- Installed apps and REST Framework settings were configured in this file.

README.md

- Added to explain project setup, API endpoints, and usage instructions.

requirements.txt

- Added to store project dependencies and package versions.

.gitignore

- Added to exclude unnecessary files such as virtual environment, cache files, and local database files from Git.

Requirement: User & Entry Tables

The assignment required two database tables:

- ➔ User
- ➔ Entry

Implementation details:

The `User` model stores:

- name
- created_date

The `Entry` model stores:

- subject
- message
- created_date
- foreign key relation to User

Technical decisions:

- `unique=True` was added to the user name field in order to prevent duplicate users.
- `db_index=True` was added to created_date fields for query optimization.
- `related_name='entries'` was used for cleaner ORM access.

Requirement: Unique User Creation

The assignment required creating a new user only when a unique name is provided.

Implementation details:

- `get_or_create()` was used in the Create Entry API.

- If the user already exists, the existing user is reused.

- This prevents duplicate user creation.

Code location:

- guestbook/views.py

Technical advantage:

- ➔ Reduces duplicate data
- ➔ Keeps database normalized
- ➔ Optimized and atomic ORM operation

Requirement: Create Entry API

Endpoint:

POST /api/entries/create/

This endpoint allows users to create a guestbook entry by sending:

- name
- subject
- message

Validation details:

- blank fields are prevented
- whitespace trimming is applied
- max length validation exists

Technical implementation:

- DRF Serializer validation
- APIView
- ORM create operation

Requirement: Get Entries API

Endpoint:

GET /api/entries/

Implemented features:

- latest entries listing
- pagination support
- 3 items per page
- descending order by created date
- username included in response

Technical implementation:

- DRF ListAPIView
- custom pagination class
- optimized queryset using select_related()

Optimization details:

- `select_related('user')` prevents N+1 query problems by retrieving related user data in a single query.

Pagination Logic

Pagination was implemented according to the assignment requirements.

Configuration:

- page_size = 3

Example:

If total entry count is 7:

- page 1 → 3 items
- page 2 → 3 items
- page 3 → 1 item

Therefore:

- count = 7
- page_size = 3
- total_pages = 3

The last page does not need to be full in pagination systems.

Requirement: Get Users Data API

Endpoint:

GET /api/users/

Implemented features:

- total message count per user
- latest entry information
- no pagination

Latest entry format:

subject | message

Technical implementation:

- ORM annotate()
- Count()
- Subquery()
- OuterRef()
- Concat()

Important note:

The assignment text explicitly required:

- **total count of messages**

However, the example response did not include this field.

Since the requirement explicitly mentioned it, the implementation includes:

- **total_messages**

This was intentionally added in order to fully satisfy the written requirements rather than relying only on the example response.

Query Optimization

Since the assignment mentioned that the data size could become large, query optimization was considered during development.

Optimizations used:

- select_related()
- annotate()
- Count()
- Subquery()
- OuterRef()
- database indexing

Reasons:

- reduce query count
- avoid N+1 query problems
- improve scalability
- move calculations to database level instead of Python loops

Automated Tests

Automated integration tests were implemented using DRF APITestCase.

The test flow includes:

- entry creation
- validation checks
- unique user creation logic
- pagination behavior
- latest entry ordering
- users endpoint response
- total message count checks
- last entry format checks

An end-to-end API flow test was also added to test the full GuestBook flow with multiple users and entries.

Test Coverage

Coverage analysis was performed using the Coverage package.

Final coverage result:

- 99% total coverage

Additional Improvements

Additional improvements implemented beyond the core requirements:

- validation enhancements
- admin panel configuration
- Black formatter integration
- requirements.txt
- README documentation
- .gitignore
- database indexing
- clean code comments

Conclusion

The GuestBook Backend API assignment was completed using Django and Django REST Framework.

All assignment requirements were implemented, including:

- entry creation
- unique user handling
- pagination
- user-based data listing
- query optimization
- automated tests

Additional improvements such as validation controls, admin panel configuration, test coverage analysis, and clean project structure were also added during development.

The project was developed with a focus on clean code, readable structure, and optimized database operations.